

Sestavte šablonu pro prioritní frontu:

```
template<class P, class D>
class PFronta { public: PFronta(unsigned);
                    ~PFronta();
                    void pridat(const P &, const D &);
                    bool odebrat(D &);
                    unsigned operator + () const; };
```

P je datový typ priority. Pro srovnání hodnot priorit je podporován operátor srovnání <. Ostatní operátory v datovém typu priority **P** nemusí být podporovány. To znamená, že při sestavení šablony jiné operátory pro srovnání priorit nelze použít.

D je datový typ dat ukládaných do fronty.

Fronta je implementována polem. Počáteční rozsah pole je dán parametrem konstruktoru.

Funkce šablony:

- **PFronta** – alokuje paměť pro uložení priorit a dat ve frontě. Počet prvků v poli je stanoven parametrem konstruktoru.
- **~PFronta** – uvolní paměť přidělenou pro pole priorit a dat.
- **pridat** – přidá do pole fronty prvek (prioritu+data). Pokud je pole plné, zvýší před přidáním prvku jeho rozsah postupem:
 - Přidělí paměť pro pole dvojnásobné velikosti, než je stávající velikost.
 - Zajistí, aby v novém poli byly uloženy stávající priority a data.
 - Paměť přidělenou pro původní pole uvolní.
- **odebrat** – odebere prvek z fronty, jeho data uloží do parametru funkce a vrátí **true**. Pokud prvek nelze odebrat, protože fronta už je prázdná, vrátí **false**. Prvek k odebrání bude vybrán dle pravidel:
 - bude odebrán prvek s nejvyšší prioritou.
 - je-li ve frontě více prvků s nejvyšší prioritou, bude z nich odebrán prvek, který je ve frontě nejdéle.
- **operator +** vrátí stávající rozsah pole pro uložení priorit a dat.

Pro přidělení a uvolnění paměti použijte operátory **new** a **delete**. Datové členy fronty budou soukromé.